Documento de Engenharia de Software

Feature Request & Product Roadmap Management System

EDS-10 — Estilo Arquitetural e Architecture Decision Records

Ano Letivo 2026/27

Índice

1 Objetivo **2**

2 Fatores que influenciam a arquitetura **2**

3 Estilo arquitetural escolhido **2**

3.1 Camadas do back-end 2

3.2 Módulos do back-end 3

3.3 Como o estilo responde aos requisitos não funcionais 3

4 Architecture Decision Records **4**

4.1 ADR-001 — Estilo arquitetural: cliente-servidor em camadas com monólito modular 4

4.2 ADR-002 — Stack tecnológica 4

4.3 ADR-003 — Integração com o Serviço de IA através de um adaptador 5

4.4 ADR-004 — Processamento assíncrono com Celery e Redis 5

4.5 ADR-005 — Autenticação com JWT e permissões por perfil 5

4.6 ADR-006 — API REST em JSON, versionada e documentada 6

4.7 ADR-007 — Atualização dos dashboards por consultas periódicas 6

5 Decisões em aberto **6**

6 Ligação aos outros artefactos **7**

1 Objetivo

Este documento define e justifica o estilo arquitetural do Feature Request & Product Roadmap Management System e regista as principais decisões de arquitetura sob a forma de ADRs (Architecture Decision Records). Cada ADR descreve o contexto, a decisão tomada, as alternativas avaliadas e as consequências, para que qualquer elemento da equipa, ou o docente, perceba porque é que o sistema está organizado desta forma.

2 Fatores que influenciam a arquitetura

A escolha do estilo arquitetural partiu dos seguintes fatores:

- **Equipa e prazo:** cinco estudantes e cerca de doze semanas de desenvolvimento, divididas em cinco sprints. A arquitetura tem de ser simples de montar, de pôr a correr e de explicar.
- **Restrições do enunciado:** o sistema tem de ter front-end e back-end, recomenda frameworks como Django e React, e tem de ter pelo menos oito interfaces além de CRUD e login.
- **Domínio com várias áreas:** o enunciado define sete áreas funcionais (gestão de pedidos, priorização, backlog, roadmap, engenharia de requisitos, colaboração e analítica), que a equipa vai desenvolver em paralelo.
- **Integração com IA:** a triagem de pedidos (US 1.2), a deteção de duplicados (US 1.5) e a geração de User Stories (US 1.3) dependem de um serviço externo de LLM, que pode ser lento ou falhar.
- **Interface rica:** o roadmap tipo Gantt (US 1.1), a ordenação do backlog por arrastar e largar e os dashboards atualizados (US 4.1) exigem uma interface interativa no browser.
- **Requisitos não funcionais:** desempenho (RNF-01), atualização dos dashboards em menos de 30 segundos (RNF-02), segurança (RNF-03), resiliência à falha da IA (RNF-07) e manutenibilidade com testes automáticos (RNF-09).

3 Estilo arquitetural escolhido

O sistema segue uma arquitetura **cliente-servidor em camadas**, com o **back-end organizado como monólito modular**:

- **Cliente:** uma Single Page Application em React corre no browser e comunica com o servidor apenas através de uma API REST em JSON.
- **Servidor:** uma única aplicação Django, instalada e executada como um todo, mas dividida internamente em módulos independentes (apps Django), um por área funcional.
- **Dados:** uma única base de dados PostgreSQL, acedida através do ORM do Django.
- **Tarefas assíncronas:** um processo Worker (Celery), com o mesmo código do back-end, executa o trabalho lento (triagem com IA e envio de e-mails), recebendo as tarefas através de uma fila Redis.
- **Integrações externas:** o Serviço de IA, o GitHub e o Serviço de E-mail são acedidos através de adaptadores próprios, isolados do resto do código.

3.1 Camadas do back-end

Dentro do back-end, cada pedido HTTP atravessa quatro camadas, e cada camada só conhece a camada imediatamente abaixo:

1. **Camada API** (views, serializers e permissões do Django REST Framework): recebe o

pedido, valida os dados de entrada, verifica a autenticação e as permissões do perfil do utilizador e devolve a resposta em JSON.
2. **Camada de serviços de domínio**: contém as regras de negócio, por exemplo `aprovar()` só com esforço e impacto definidos, ou `estaPronta()` para a Definition of Ready.
3. **Camada de acesso a dados** (modelos e ORM do Django): lê e grava na base de dados.
4. **Adaptadores de integração**: encapsulam as chamadas ao LLM, ao servidor de e-mail e aos webhooks do GitHub.

3.2 Módulos do back-end

Os módulos correspondem às áreas do Product Backlog e às classes do diagrama de classes (EDS-13):

- **contas:** utilizadores, perfis e equipas;
- **pedidos:** pedidos, categorias, etiquetas (tags), anexos, triagem e fusão de duplicados;
- **priorizacao:** votos, esforço e impacto, aprovação;
- **backlog:** Product Backlog, sprints e Definition of Ready;
- **requisitos:** User Stories, critérios de aceitação e rastreabilidade;
- **roadmap:** releases, milestones e Visão Executiva;
- **colaboracao:** comentários, menções e notificações;
- **analitica:** Lead Time, Cycle Time, Velocity e capacidade;
- **integracao_ia** e **integracao_github:** adaptadores para os sistemas externos.

Regra de dependências: um módulo só usa outro através das suas funções de serviço, nunca acedendo diretamente aos seus modelos internos. Assim, cada elemento da equipa pode trabalhar num módulo sem interferir nos restantes, e um módulo pode, no futuro, ser extraído para um serviço separado.

3.3 Como o estilo responde aos requisitos não funcionais

- **Manutenibilidade e testes (RNF-09):** as regras de negócio estão na camada de serviços, separadas do HTTP e da base de dados, por isso podem ser testadas com testes unitários simples.
- **Resiliência (RNF-07):** a IA só é chamada através do adaptador, com tempo limite e alternativa sem IA; a triagem corre no Worker, por isso uma falha da IA nunca bloqueia a submissão de um pedido.
- **Desempenho (RNF-01):** o trabalho lento sai do caminho do pedido HTTP; as consultas de métricas usam índices nas datas de estado.
- **Segurança (RNF-03):** autenticação centralizada na camada API (JWT) e permissões verificadas por perfil em todos os endpoints.
- **Atualização dos dashboards (RNF-02):** a SPA volta a pedir os dados de métricas em intervalos curtos (ADR-007).

Os diagramas de contentores (C4 nível 2) e de componentes do back-end (C4 nível 3) estão a seguir aos ADRs.

4 Architecture Decision Records

4.1 ADR-001 — Estilo arquitetural: cliente-servidor em camadas com monólito modular

Estado: Aceite.

Contexto. O sistema tem sete áreas funcionais desenvolvidas em paralelo por cinco pessoas, em doze semanas, e tem de ser apresentado e explicado a cada sprint. A equipa não tem experiência a operar sistemas distribuídos.

Decisão. Adotar uma arquitetura cliente-servidor: uma SPA no browser e um único back-end, organizado em camadas (API, serviços, dados, integrações) e dividido em módulos por área funcional, com uma única base de dados.

Alternativas consideradas. *Microserviços*, um serviço por área funcional: rejeitado, porque exigiria várias bases de dados, comunicação pela rede, orquestração de contentores e monitorização distribuída, sem benefício real para o volume de utilizadores deste projeto. *Monólito MVC com páginas geradas no servidor* (templates Django): rejeitado, porque o roadmap Gantt, a ordenação por arrastar e largar e os dashboards dinâmicos exigem uma interface rica no browser. *Serverless ou Backend-as-a-Service* (por exemplo Firebase): rejeitado, porque limita as regras de negócio complexas (Definition of Ready, transições de estado) e a modelação relacional.

Consequências positivas. Um único projeto para instalar, testar e implementar; transações simples numa só base de dados; fronteiras claras entre módulos que permitem trabalho em paralelo; possibilidade de extrair um módulo para um serviço no futuro.

Consequências negativas e riscos. Todo o back-end é implementado como um todo, por isso um erro grave num módulo pode afetar o sistema inteiro; escalar exige escalar a aplicação toda. A separação entre módulos depende da disciplina da equipa e tem de ser verificada nas revisões de código.

4.2 ADR-002 — Stack tecnológica

Estado: Aceite.

Contexto. O enunciado recomenda frameworks modernas como Django, React ou Node.js. A equipa precisa de autenticação, painel de administração e ORM sem os implementar de raiz.

Decisão. Front-end em React com Vite; back-end em Python com Django e Django REST Framework; base de dados PostgreSQL; ambiente de desenvolvimento com Docker Compose; repositório em GitHub com integração contínua em GitHub Actions.

Alternativas consideradas. *Node.js com Express*: mais flexível, mas sem ORM, autenticação e administração incluídos. *FastAPI*: moderna e rápida, mas exigiria escolher e integrar à parte ORM, migrações e administração. *MySQL ou SQLite*: o SQLite não suporta bem acessos concorrentes; o PostgreSQL tem melhor suporte para restrições (`CHECK`, chaves compostas) usadas no modelo ER (EDS-14).

Consequências positivas. Django traz ORM, migrações, autenticação e painel de administração; React tem um ecossistema grande de componentes (gráficos, Gantt, arrastar e largar); Docker Compose põe o sistema a correr com um único comando em qualquer computador da equipa.

Consequências negativas e riscos. Duas linguagens (Python e JavaScript) no mesmo projeto; quem trabalha nos dois lados tem de conhecer ambas.

4.3 ADR-003 — Integração com o Serviço de IA através de um adaptador

Estado: Aceite.

Contexto. A triagem (US 1.2), a deteção de duplicados (US 1.5) e a geração de User Stories (US 1.3) usam um LLM externo. Estes serviços podem ser lentos, falhar, mudar de fornecedor ou ter custos por utilização.

Decisão. Todo o acesso ao LLM passa por um único módulo, `integracao_ia`, que constrói os prompts, aplica um tempo limite, valida o formato da resposta e devolve uma alternativa sem IA quando a chamada falha. A IA apenas sugere: nenhuma sugestão é aplicada sem confirmação do Gestor de Produto.

Alternativas consideradas. *Chamar a API do LLM diretamente em cada módulo*: rejeitado, porque espalharia prompts e tratamento de erros por todo o código e tornaria a troca de fornecedor difícil. *Modelo de IA local*: rejeitado, por exigir hardware e conhecimentos fora do âmbito do projeto.

Consequências positivas. Trocar de fornecedor de LLM só altera um módulo; nos testes, o adaptador pode ser substituído por uma versão falsa; o comportamento em caso de falha é igual em todo o sistema (RNF-07).

Consequências negativas e riscos. As sugestões dependem da qualidade do prompt e do modelo; é preciso gerir a chave de acesso como segredo e controlar o custo das chamadas.

4.4 ADR-004 — Processamento assíncrono com Celery e Redis

Estado: Aceite.

Contexto. A triagem com IA pode demorar vários segundos e o envio de e-mails depende de um servidor externo. Se fossem feitos dentro do pedido HTTP, o cliente ficaria à espera ou receberia erros quando estes serviços falhassem.

Decisão. Executar a triagem e o envio de e-mails como tarefas assíncronas num processo Worker com Celery, usando Redis como fila de mensagens. A API coloca a tarefa na fila e responde de imediato; o Worker processa-a, com novas tentativas em caso de falha. A geração de User Story (US 1.3) continua síncrona, porque o Gestor de Produto espera pelo esboço para o rever.

Alternativas consideradas. *Fazer tudo dentro do pedido HTTP*: rejeitado, pelas esperas e falhas descritas. *Fila guardada na própria base de dados*: mais simples, mas com pior desempenho e menos suporte para novas tentativas.

Consequências positivas. A submissão de pedidos é rápida e nunca falha por causa da IA ou do e-mail; as tarefas falhadas são repetidas automaticamente.

Consequências negativas e riscos. Mais dois contentores para pôr a correr (Worker e Redis), já incluídos no Docker Compose e no diagrama C4 nível 2; os erros nas tarefas têm de ser registados e consultados à parte.

4.5 ADR-005 — Autenticação com JWT e permissões por perfil

Estado: Aceite.

Contexto. A SPA e a API são aplicações separadas que comunicam por HTTP. Existem quatro perfis (Cliente, Gestor de Produto, Programador e Gestor de Projeto) com acessos diferentes.

Decisão. Autenticação por tokens JWT (biblioteca `djangorestframework-simplejwt`), com token de acesso de curta duração e token de renovação. As permissões são verificadas na camada API com base no atributo `perfil` do Utilizador. As palavras-passe são guardadas com hash.

Alternativas consideradas. *Sessões com cookies do Django*: funcionam bem quando o front-end é servido pelo próprio Django, mas complicam a separação entre SPA e API. *Fornecedor externo de identidade* (OAuth com Google ou GitHub): adiado para sprints futuros, por acrescentar configuração sem ser exigido.

Consequências positivas. A API não guarda estado de sessão; as regras de acesso estão num só sítio e são testáveis.

Consequências negativas e riscos. É preciso tratar a renovação e a expiração dos tokens na SPA e guardá-los de forma segura no browser.

4.6 ADR-006 — API REST em JSON, versionada e documentada

Estado: Aceite.

Contexto. O front-end e o back-end são desenvolvidos em paralelo por pessoas diferentes, que precisam de um contrato estável entre os dois.

Decisão. Expor uma API REST em JSON com o prefixo `/api/v1/`, recursos nomeados em português de acordo com o diagrama de classes (`/pedidos`, `/user-stories`, `/sprints`), e documentação OpenAPI gerada automaticamente (`drf-spectacular`).

Alternativas consideradas. *GraphQL*: mais flexível nas consultas, mas com curva de aprendizagem maior e menos integrado com o Django REST Framework.

Consequências positivas. Os endpoints são os mesmos que aparecem nos diagramas de sequência; a documentação gerada serve de contrato entre front-end e back-end e pode ser testada no browser.

Consequências negativas e riscos. Algumas páginas, como os dashboards, podem precisar de vários pedidos à API ou de endpoints de agregação próprios.

4.7 ADR-007 — Atualização dos dashboards por consultas periódicas

Estado: Aceite.

Contexto. O RNF-02 exige que os dashboards reflitam alterações em menos de 30 segundos, sem recarregar a página (US 4.1).

Decisão. A SPA volta a pedir os dados das métricas à API a cada 15 segundos enquanto o dashboard está aberto (polling).

Alternativas consideradas. *WebSockets com Django Channels*: atualização instantânea, mas exige um servidor assíncrono adicional e mais complexidade para um ganho que o requisito não pede.

Consequências positivas. Implementação simples com o mesmo mecanismo REST usado no resto do sistema; cumpre o limite de 30 segundos.

Consequências negativas e riscos. Gera pedidos repetidos ao servidor mesmo sem alterações; se o número de utilizadores crescer, esta decisão deve ser revista em favor de WebSockets.

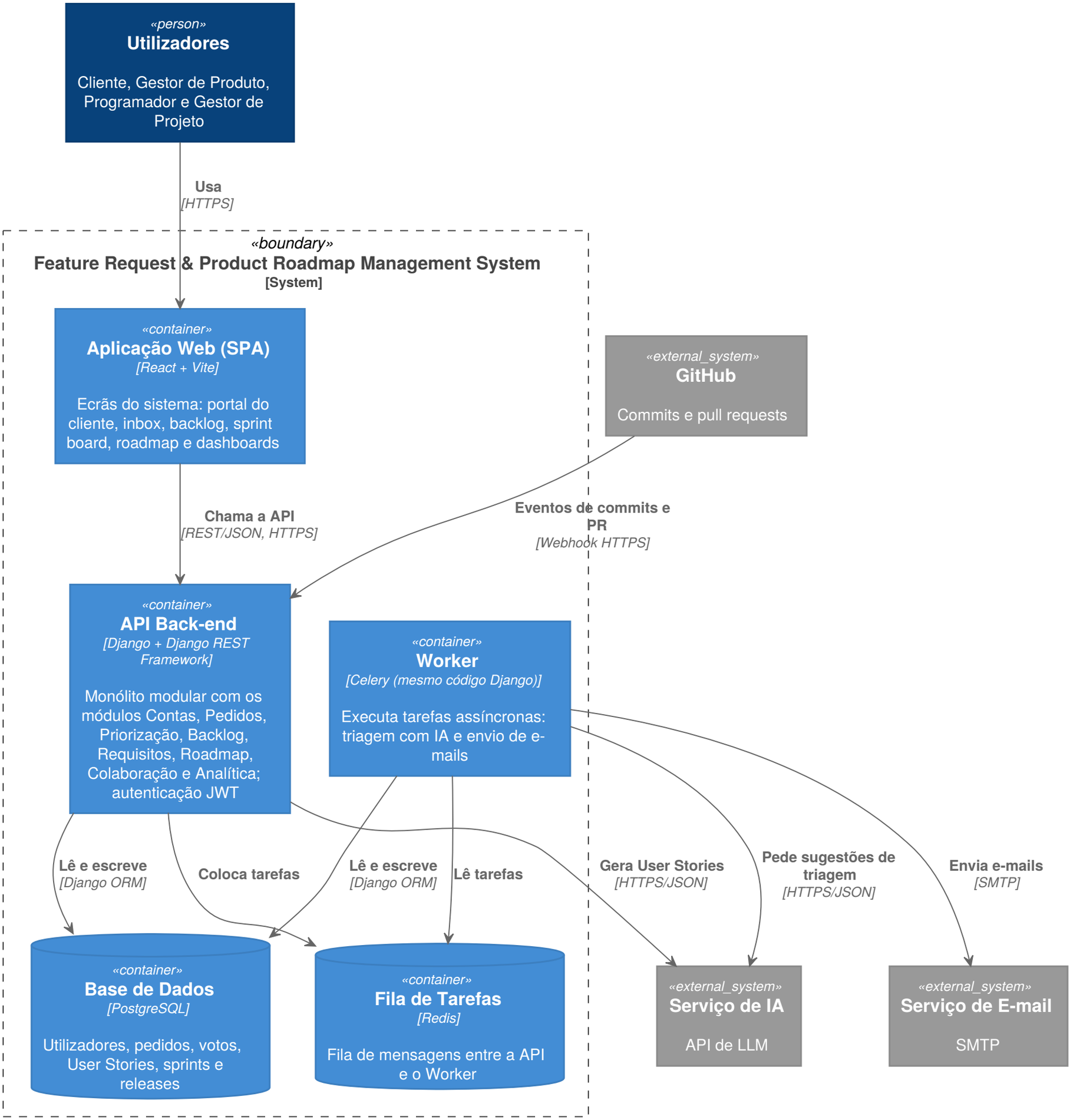
5 Decisões em aberto

- **Ferramenta de IA para programação** (restrição 7 do enunciado): a escolha e o modo de utilização devem ser registados num novo ADR quando a equipa os definir.
- **Alojamento da versão final**: a definir até ao Sprint 5, quando for preparada a implementação final.

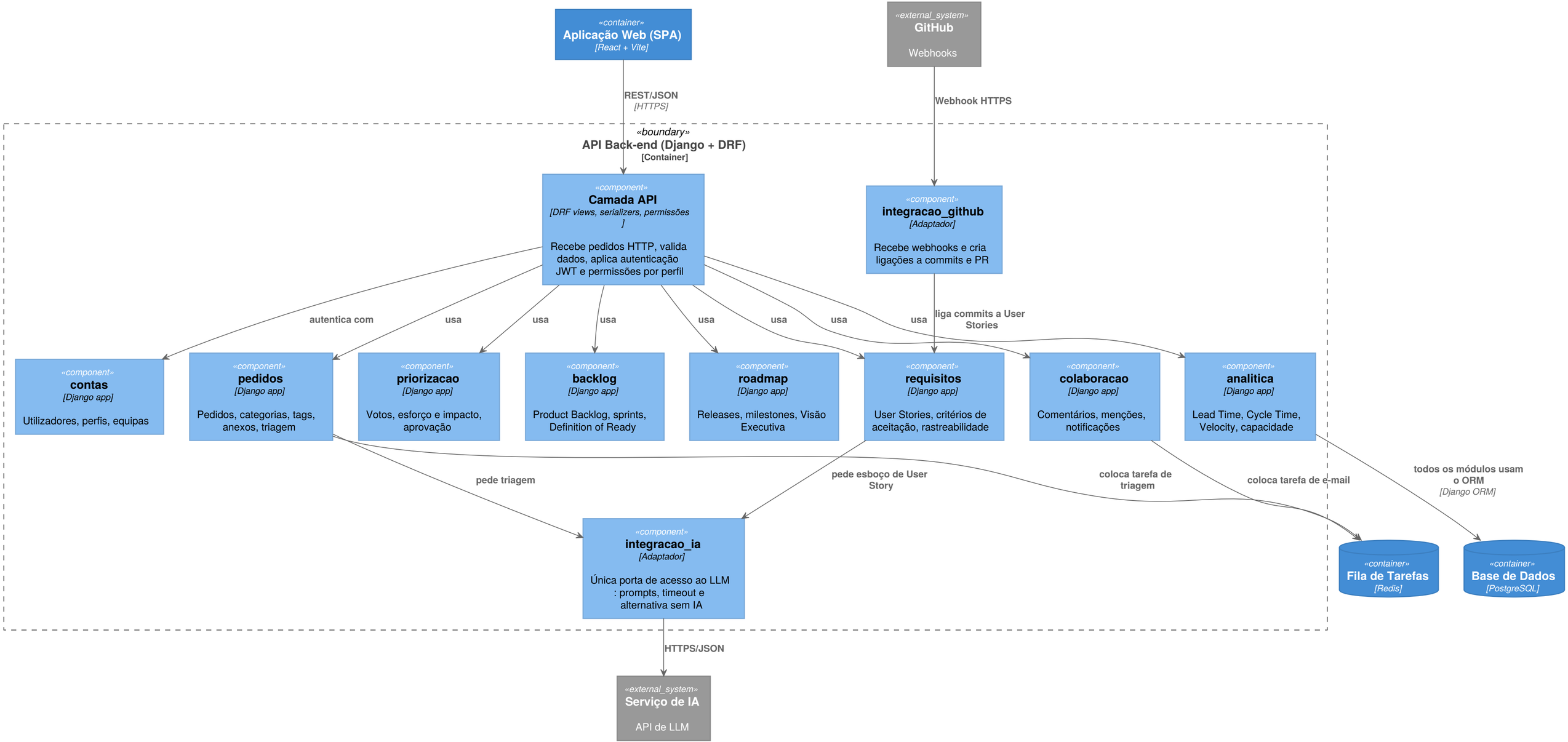
6 Ligação aos outros artefactos

- **Diagramas C4 (EDS-11):** o nível 2 mostra os contentores aqui descritos, incluindo o Worker e a Fila de Tarefas.
- **Diagrama de classes (EDS-13) e modelo ER (EDS-14):** cada módulo do back-end agrupa as classes e tabelas da sua área.
- **Diagramas de sequência:** os participantes (Aplicação Web, API Back-end, Base de Dados, Serviço de IA e Serviço de E-mail) são os contentores desta arquitetura, e os endpoints seguem o ADR-006.

C4 Nível 2 — Diagrama de Contentores



Estrutura do Back-end — Camadas e Módulos (C4 Nível 3)



Código PlantUML

c4_nivel2_contentores.puml

```
@startuml c4_nivel2_contentores
!include <C4/C4_Container>
skinparam dpi 96
skinparam defaultFontName Helvetica
title C4 Nível 2 -- Diagrama de Contentores

Person(utilizadores, "Utilizadores", "Cliente, Gestor de Produto, Programador e Gestor de Projeto")

System_Boundary(sistema, "Feature Request & Product Roadmap Management System") {
  Container(spa, "Aplicação Web (SPA)", "React + Vite", "Ecrãs do sistema: portal do cliente, inbox, backlog, sprint board, roadmap e dashboards")
  Container(api, "API Back-end", "Django + Django REST Framework", "Monólito modular com os módulos Contas, Pedidos, Priorização, Backlog, Requisitos, Roadmap, Colaboração e Analítica; autenticação JWT")
  Container(worker, "Worker", "Celery (mesmo código Django)", "Executa tarefas assíncronas: triagem com IA e envio de e-mails")
  ContainerDb(redis, "Fila de Tarefas", "Redis", "Fila de mensagens entre a API e o Worker")
  ContainerDb(db, "Base de Dados", "PostgreSQL", "Utilizadores, pedidos, votos, User Stories, sprints e releases")
}

System_Ext(ia, "Serviço de IA", "API de LLM")
System_Ext(github, "GitHub", "Commits e pull requests")
System_Ext(email, "Serviço de E-mail", "SMTP")

Rel(utilizadores, spa, "Usa", "HTTPS")
Rel(spa, api, "Chama a API", "REST/JSON, HTTPS")
Rel(api, db, "Lê e escreve", "Django ORM")
Rel(api, redis, "Coloca tarefas")
Rel(worker, redis, "Lê tarefas")
Rel(worker, db, "Lê e escreve", "Django ORM")
Rel(worker, ia, "Pede sugestões de triagem", "HTTPS/JSON")
Rel(api, ia, "Gera User Stories", "HTTPS/JSON")
Rel(worker, email, "Envia e-mails", "SMTP")
Rel(github, api, "Eventos de commits e PR", "Webhook HTTPS")
@enduml
```

componentes_backend.puml

```
@startuml componentes_backend
!include <C4/C4_Component>
skinparam dpi 96
skinparam defaultFontName Helvetica
title Estrutura do Back-end -- Camadas e Módulos (C4 Nível 3)

Container(spa, "Aplicação Web (SPA)", "React + Vite")
ContainerDb(db, "Base de Dados", "PostgreSQL")
ContainerDb(redis, "Fila de Tarefas", "Redis")
System_Ext(ia, "Serviço de IA", "API de LLM")
System_Ext(github, "GitHub", "Webhooks")

Container_Boundary(api, "API Back-end (Django + DRF)") {
  Component(rest, "Camada API", "DRF views, serializers, permissões", "Recebe pedidos HTTP, valida dados, aplica autenticação JWT e permissões por perfil")
  Component(contas, "contas", "Django app", "Utilizadores, perfis, equipas")
  Component(pedidos, "pedidos", "Django app", "Pedidos, categorias, tags, anexos, triagem")
```

```
  Component(prior, "priorizacao", "Django app", "Votos, esforço e impacto, aprovação")
  Component(backlog, "backlog", "Django app", "Product Backlog, sprints, Definition of Ready")
  Component(req, "requisitos", "Django app", "User Stories, critérios de aceitação,
    rastreabilidade")
  Component(roadmap, "roadmap", "Django app", "Releases, milestones, Visão Executiva")
  Component(colab, "colaboracao", "Django app", "Comentários, menções, notificações")
  Component(anal, "analitica", "Django app", "Lead Time, Cycle Time, Velocity, capacidade")
  Component(iaad, "integracao_ia", "Adaptador", "Única porta de acesso ao LLM: prompts, timeout
    e alternativa sem IA")
  Component(ghad, "integracao_github", "Adaptador", "Recebe webhooks e cria ligações a commits e
     PR")
}

Rel(spa, rest, "REST/JSON", "HTTPS")
Rel(rest, pedidos, "usa")
Rel(rest, prior, "usa")
Rel(rest, backlog, "usa")
Rel(rest, req, "usa")
Rel(rest, roadmap, "usa")
Rel(rest, anal, "usa")
Rel(rest, colab, "usa")
Rel(rest, contas, "autentica com")
Rel(pedidos, iaad, "pede triagem")
Rel(req, iaad, "pede esboço de User Story")
Rel(iaad, ia, "HTTPS/JSON")
Rel(pedidos, redis, "coloca tarefa de triagem")
Rel(colab, redis, "coloca tarefa de e-mail")
Rel(github, ghad, "Webhook HTTPS")
Rel(ghad, req, "liga commits a User Stories")
Rel_D(anal, db, "todos os módulos usam o ORM", "Django ORM")
@enduml
```